Re-run Before You Roll Back: A Pre-registered Audit of Same-Family Model Upgrades for Tool-Using Agents

Lakshmi Charan Lingisetty
Independent Researcher, Louisville, Kentucky, USA
lingisettycharan123@gmail.com

ABSTRACT

When a provider retires the model behind an agent, teams switch to the next model in the same family and ask which of their workflows will break. I answer this question with a pre-registered, repeat-aware audit of six same-family upgrade pairs from four families, including a provider's two official retirement replacements. Each model ran every task three times on three executable, code-graded testbeds with side effects: a clinical agent on a FHIR health record, an operations agent on six real services, and an MLOps agent that diagnoses production degradation by re-executing pipelines. Mean success rose in 15 of 18 pair–testbed cells and fell significantly in none. Of the tasks a one-shot regression suite flags as broken, 63% (95% CI 50–76%) pass again on re-run. Re-running each failure up to twice removes these false alarms for 20% more episodes, and a first-action probing heuristic did not do better. Reliable regressions (always passed before, always failed after) affect 4.5% of previously reliable tasks, and none survives correction for multiple testing. Unlicensed writes fell significantly in five of six pairs and rose significantly in none. Five of my six hypotheses (five pre-registered, one added by a documented amendment) were not supported, and I report all of them. Finally, a ten-hour outage of the clinical server during my own collection silently corrupted 63% of clinical episodes. On that data, three upgrades appeared to lower success and an official replacement model appeared to reliably break a clinical workflow; checking the environment on every episode removed both artifacts. I release all episodes, serving-provider logs, the per-episode environment check and the analysis code.

1. Introduction

Teams that run AI agents in production change the underlying model on a schedule they do not control. Providers retire models within months of release, and the recommended replacement is usually a newer member of the same family that scores higher on public benchmarks. The engineering question at that moment is narrow and practical: *which of the workflows that work today will stop working after the switch?* A higher average score does not answer it. An upgrade can raise mean success while breaking a particular workflow, a pattern known from classifier updates as negative flips (Yan et al. 2021; Xie et al. 2021) and recently documented for single-turn LLM benchmarks (Cacioli 2026; Sheng and Lu 2026).

For agents the question is harder for two reasons. First, agents are noisy. The same model on the same task can succeed in one run and fail in the next, even at temperature zero, and run-to-run variation in agent benchmarks can be as large as the gap between model versions (Ariño de la Rubia and Pafka 2026; Yagubyan 2026). A regression suite that runs each task once will report "regressions" that are only noise. Second, an agent's failures are actions with side effects, taken in an environment that can itself fail. A regressed agent may place an order nobody asked for; an agent facing a broken environment fails for reasons that have nothing to do with the model.

I ask what actually changes when an agent's model is upgraded to the next version of the same family, and answer with a pre-registered, repeat-aware audit. Six upgrade pairs from four families, including a provider's two official retirement replacements, run every task three times on three executable testbeds whose grading reads only the resulting state: a clinical agent that reads and writes a FHIR health record (MedAgentBench tasks (Jiang et al. 2025) plus hazard variants), an operations agent acting on real services (PostgreSQL, Gitea, Mailpit, Redis, an S3 store and a ledger database), and an MLOps agent that diagnoses production model degradation by re-executing pipeline components. A within-task permutation test separates *reliable* negative flips (old model passes all runs, new model fails all runs) from flips that same-model repetition produces by chance.

The findings and contributions of this paper are:

1. **One-shot regression reports are at least half noise.** Of the tasks a single run flags as regressed, 63% (95% CI 50–76%) pass again on re-run. Re-running each failure up to twice before flagging it removes these false alarms for about a fifth more episodes, and a first-action probing heuristic I designed does not beat this simple rule.
2. **Reliable regressions are rare and, at this scale, not separable from churn.** Across 18 pair–testbed cells, fewer than one in twenty tasks that the old model always solved is always failed by the new one, and no cell's count of reliable negative flips survives correction for multiple testing. Same-family upgrades in my sample are better on average and, as far as three runs can tell, not broken in places.
3. **Newer generations take fewer unlicensed actions.** Writes outside the licensed set fall significantly in five of six pairs and rise significantly in none, contrary to the mixed picture my pilot suggested.
4. **A silent environment failure can manufacture every effect I looked for.** A ten-hour outage of the clinical server during my own collection corrupted 63% of clinical episodes, and 43% of those were still graded as successes, mostly because doing nothing is correct on many conditional tasks. On that data, three upgrades appeared to *lower* success, the

provider's official replacement appeared to *reliably break* a vital-sign workflow, and reliability appeared to erode; all three disappeared on clean data. I describe how the failure evaded a per-job health check and release the per-episode check and scan that caught it.

All hypotheses, task sets and analysis code were frozen before the confirmatory runs; I report every pre-registered test, including the five of six (one added by amendment) whose criteria were not met, and list every amendment and deviation (Appendix A). Every episode, the serving provider of every call and the analysis code are released.

2. Related Work

Regression under model updates. Backward compatibility of model updates was first framed for human–AI teams, where a more accurate update can still break the user's mental model of when the system is right (Bansal et al. 2019). *Negative flips*, instances the old model handled and the new one does not, were defined for image classifiers (Yan et al. 2021) and NLP models (Xie et al. 2021), and reduced through training-time constraints, distillation or gated fusion of old and new predictions (Yan et al. 2021; Lai et al. 2023; Zhu et al. 2024). For large language models, MUSCLE showed that task adapters suffer negative flips when the base model changes (Echterhoff et al. 2024), and API snapshots of the same commercial model were found to drift substantially within months (Chen, Zaharia, and Zou 2024). Two 2026 studies are closest to this work. Cacioli (2026) applies repeated sampling and a reliable-change index to multiple-choice benchmarks and finds that single-shot evaluation misclassifies many items as changed. Sheng and Lu (2026) study sample-level regressions across six same-family upgrade pairs and find that no single signal predicts which samples will regress. Both work with single-turn questions. I study multi-step agents whose failures are actions on real systems, separate reliable flips from run-to-run churn with a permutation test, and classify what kind of action changed.

Migrating applications across models. Regression testing of applications built on evolving LLM APIs was argued for by Ma, Yang, and Kästner (2024), and supported by interactive tooling that diffs old and new outputs (Dixit et al. 2024) and by regression suites for code models (Abbassi et al. 2025). Prompt-transfer methods adapt prompts from one model to another (Wang et al. 2025). A mining study of open-source applications found that most migrations off retired models happen after the shutdown date (Kim 2026), and governance proposals call for compatibility gates before updates reach production (Chishti, Oyinloye, and Li 2026). These works motivate my question but do not measure agent-level behaviour under repeated execution.

Reliability and noise in agent evaluation. Agent benchmarks report large run-to-run variation: τ-bench introduced pass^k (Yao et al. 2025), Rabanser et al. (2026) propose a family of reliability

metrics, and recent measurements show that tool-call sequences and arguments diverge across identical runs (Yagubyan 2026) and that within-configuration noise can exceed the gap between model versions (Ariño de la Rubia and Pafka 2026). Statistical tools for comparing models efficiently include paired tests for degradation (Kübler et al. 2026), active testing (Kossen et al. 2021) and anytime-valid sequential evaluation (Hsu and Shekhar 2026; Arviv et al. 2026); PACE applies sequential testing to accepting self-modifications of an agent (Shawn 2026). Process-level evaluation of evolving enterprise agent skills across model changes appeared concurrently (Vo, Doshi, and Sheinin 2026); it diagnoses deviations on two skills without old-to-new pairs or noise control. My contribution is complementary: a repeat-aware audit of real same-family upgrades on three executable testbeds, which quantifies how much of what a one-shot regression suite reports is noise.

Unrequested actions by agents. Agents that act beyond their mandate have been studied as over-authorization in tool calling (Zhang et al. 2026), out-of-scope actions by coding agents (Qu et al. 2026) and scope adherence in offensive-security agents (Caldwell et al. 2026); mitigations derive permissions or policies from the user's request (Shi et al. 2025). These works compare models at one point in time. I measure how unlicensed side-effecting writes change *across generations of the same family*, on tasks whose licence depends on the state of the world (an order is correct only if a lab value meets a condition).

3. Study Design

3.1 Upgrade pairs

An *upgrade pair* is an older and a newer model from the same family that a team would plausibly migrate between. I study six pairs from four families (Table 1): two official retirement replacements published by a provider (each older small model is retired in favour of the same newer one), the step between those two older models, and the successive releases of three open-weight families. A seventh, pre-registered pair of two Gemini Flash-Lite versions ran on a free tier as a best-effort addition. Its daily quota of 50 requests per model made it impossible to replace its episodes lost to the outage described in Section 5, so it is excluded from all analyses.

Table 1. Upgrade pairs.

Pair	Old model	New model	Relation
P1	GPT-4.1 nano	GPT-5.6 Luna	provider's official replacement
P2	GPT-5 nano	GPT-5.6 Luna	provider's official replacement
P3	GPT-4.1 nano	GPT-5 nano	successive small models
P4	Qwen3-30B-A3B (2507)	Qwen3.5-35B-A3B	successive open-weight MoE releases
P5	Mistral Small 3.2	Mistral Small (2603)	successive releases
P6	Gemma 3 27B	Gemma 4 31B	successive open-weight releases

All models are reached through one routing service. To keep a model's behaviour from depending on which host answered, every model is pinned to a single serving provider with fallbacks disabled, and the provider that served each call is logged. All models run at temperature 0 with the same prompts, tools, round limits and response truncation, and with the cheapest reasoning setting where a model has one.

3.2 Testbeds

All three testbeds are executable and graded by code that reads the resulting state; no model judges any outcome.

Clinical (100 tasks). The agent answers questions about patients and places orders through the FHIR API of an electronic health record server, using MedAgentBench task instances (Jiang et al. 2025) and hazard variants (unfamiliar API aliases and value drift, an allergy, renal impairment, a duplicate order already on file, and a look-alike patient). Sixty-four of the 100 tasks are conditional: an order is licensed only if a lab value meets a threshold, or only if no recent result exists. A reference solver computes from the live server state the answer and the set of licensed orders for every task. An episode succeeds when the final answer is correct, every required order is placed, and no unlicensed order is placed. Write requests are recorded and acknowledged but not applied, so every episode sees the same record.

Operations (50 tasks). The agent completes short workflows (open an issue and comment on it, send a notification, record inventory and ledger movements, store an object) on real services running in containers: PostgreSQL, Gitea, Mailpit, Redis, an S3-compatible store and TigerBeetle. Every episode gets its own namespace. Grading reads only service-side records: the required effects must be present exactly once with the right content, and a scan of the namespace finds any extra or collateral effect. The agent must also report its outcome truthfully.

MLOps (40 cases). The agent is on call for a production model whose performance has dropped. It may roll back or replay pipeline components, measure the effect of a change and test the time boundary of a shift, within a budget of pipeline executions, and must then decide which

components to fix and whether to retrain. The oracle comes from re-executing the real pipelines on public datasets from five domains with injected defects, genuine drift, both or neither.

The confirmatory task sets are disjoint from the pilot and smoke sets; the operations and MLOps sets are seeded stratified subsamples of larger pools.

3.3 Measurements

For each pair and testbed, every task is run times by each model, each run an independent model call. Episodes in which the model API failed, or in which the environment returned a connection error to the agent, are not graded but re-run. A task enters a pair's analysis only when both models have all three runs. Let be the success of the old and new model on task in run .

- **Single-run regression:** and among tasks the old model passes in all runs. This is what a one-shot regression suite reports.
- **Reliable negative flip (RNF):** and . Reliable positive flips are defined symmetrically.
- **Noise test.** Under the null hypothesis that a task's runs are exchangeable between the two models, I permute the run labels within each task (10,000 permutations) and compare the observed RNF count with its permutation distribution (Good 2005). Holm's correction (Holm 1979) is applied across all pair–testbed cells.
- **pass^3 and erosion.** pass^3 is the share of tasks passed in all three runs (Yao et al. 2025). The erosion statistic of a cell is the number of tasks always passed by the old model but not the new one, minus the reverse, tested one-sided with the same within-task permutation (4,000 permutations) and Holm's correction within each testbed.
- **Failure categories** of the new model on RNF tasks: interface (malformed actions, round limit), unrequested write, wrong write (a requested write with wrong content or duplicated), missing write, wrong answer, and answer format (the right value in a shape the task's output contract rejects).
- **Unlicensed writes (overreach):** a write outside the licensed set. In the clinical testbed it is measured on the tasks whose licensed set is empty; in the operations testbed it is any extra or collateral effect found by the scan. Per pair, per-task rates (mean over three runs) are compared with a two-sided Wilcoxon signed-rank test, Holm-corrected across pairs.
- **Triage procedures** for finding RNFs: *one-shot* (run once, flag every failure), *re-run failures* (re-run each failure up to twice, flag only 0/3), and *first-action probing* (run only the new model's first step on every task; fully check tasks whose first action differs from the old model's usual one, plus a random audit of ten others), compared by episodes spent (a probe

costs the measured fraction of an episode), flips found and false alarms, averaged over 200 random seeds.

4. Results

All numbers below are computed on the repaired data (Section 5) by the frozen analysis scripts. Six pairs × three testbeds give 18 cells. One pair lost 4 of its 100 clinical tasks to provider rate limits that persisted through three re-run passes; those tasks are excluded from that pair's clinical cell, and every other cell is complete. Table 2 summarises the pre-registered tests; Table 4 in Appendix C gives every cell.

Table 2. Pre-registered hypotheses and outcomes on the repaired data.

Hypothesis (pre-registered criterion)	Result	Verdict
H1: of single-run regressions, > 50% pass on re-run (CI lower bound > 0.5)	34 of 54 (63%), 95% CI [0.50, 0.76]	not supported (bound = 0.50)
H2: ≥ 1 cell with significant RNF, and pooled RNF rate < 10% (upper bound)	0 cells significant; 20 of 449 (4.5%), upper bound 6.8%	not supported (rare, but none significant)
H3: ≥ 2 pairs change unlicensed writes significantly, not all in one direction	5 of 6 pairs significant, all decreases	not supported (one direction only)
H4: re-running failures costs ≤ 25% more episodes	+88 of 449 (19.6%)	supported
H5: ≥ 2 improving pairs with significant RNF	0	not supported
H6: ≥ 2 cells with significant erosion in Ops or MLOps	0 in Ops, 0 in MLOps	not supported

4.1 Upgrades are better on average

Every new model is at least as good as its predecessor on average in almost every cell (Figure 1). Mean success rises in 15 of 18 cells, and its 95% bootstrap interval lies above zero in 12; no interval lies below zero. The gains are largest on the operations testbed, where both open-weight successors go from failing most workflows to passing nearly all of them (Gemma 0.13 → 0.93, Qwen 0.29 → 0.96). Almost all of the older Qwen model's failures there are interface errors (malformed or unusable tool calls), while the older Gemma model's are a mix of wrong outcome reports, wrong writes and interface errors. On the clinical testbed the provider's replacement for its oldest small model gains 18 points (0.45 → 0.63). The only point decreases are small and uncertain: one OpenAI step (−0.03) and the Qwen step (−0.08) on MLOps, and the Mistral step on the clinical testbed (−0.003).

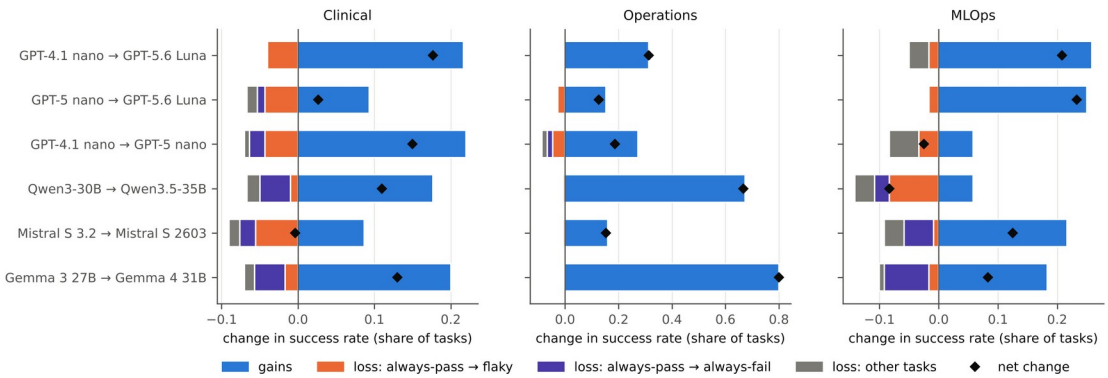


Figure 1. Where each upgrade's change in mean success comes from. Bars to the right are gains; bars to the left split the losses into tasks the old model always passed that became flaky (1–2 of 3), tasks it always passed that the new model always fails (reliable negative flips), and losses on other tasks. Diamonds mark the net change.

4.2 At least half of one-shot regression reports are false alarms

A one-shot regression suite runs each task once with the new model and flags every task the old model handled that now fails. In my data that procedure flags 54 tasks across the 18 cells (counting only tasks the old model passed in all three runs, so that the old model's own flakiness cannot cause the flag). Of these, 34 (63%) pass in at least one of the new model's next two runs (Figure 2); only 20 fail all three times. The pre-registered test required the lower end of the 95% interval to exceed one half; it equals 0.50, so H1 is formally not supported, but the point estimate and the interval both say that at least half of what a one-shot suite reports is run-to-run noise.

The rule of re-running each failure up to twice before flagging it finds all 20 reliable flips with no false alarms, at a cost of 88 extra episodes (19.6% more than one shot; H4). As a secondary analysis I pre-registered a first-action probing heuristic: it runs only the new model's first step on every task, fully checks the tasks whose first action differs from the old model's usual one plus a small random audit of the rest, and bounds the flips it may have missed. Over 200 random seeds it cost more (670 episodes on average against 537) and missed on average 4.8 of the 20 flips, because many reliable flips begin with the same first action as the old model. Confirming tasks in random order under the probe's budget (capped at what checking every task costs) found 17 of the 20 on average. The simple rule is the best of the procedures I tried.

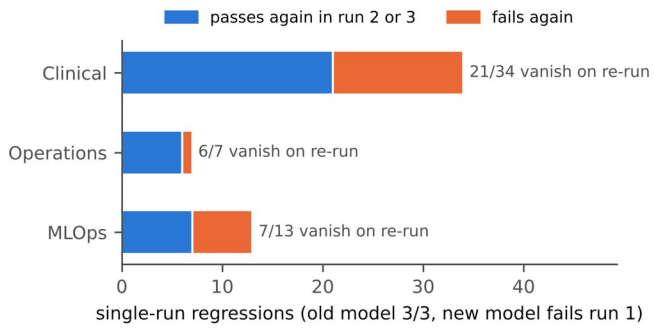


Figure 2. Single-run regressions per testbed and whether they persist when the new model is run twice more.

4.3 Reliable regressions are rare and not separable from churn

Pooled over cells, 20 of the 449 tasks that an old model always solved are always failed by its successor (4.5%; 95% upper bound 6.8%). Reliable positive flips outnumber them six to one (129). No cell's RNF count is significant after Holm correction across the 18 cells; the smallest corrected p-value is 0.088 (Gemma on the clinical testbed, 4 RNF where 0.75 are expected under exchangeability). Five cells are below 0.05 before correction: Gemma and Qwen on the clinical testbed (4 RNF each), Gemma and Mistral on MLOps (3 and 2), and Mistral on the clinical testbed (2). Several of these concentrate on one task template. Three of Qwen3.5's four clinical RNFs are instances of one template (report the last HbA1c value and its date, and order a new test if it is older than a year; 12 instances in the set), and two of Gemma 4's four are instances of another (report the most recent capillary blood glucose; 4 instances). A team whose workflow is one of these templates would experience a real regression. But with three runs and 18 cells, my data cannot distinguish them from the churn that repeated runs of a single model produce, and H5, which required such breaks in improving pairs, is not supported.

Of the 20 RNFs, 9 are unrequested writes, 6 wrong answers, 2 wrong writes, 2 answer-format failures and 1 missing write. None is an interface failure.

4.4 Newer generations take fewer unlicensed actions

Unlicensed writes fall in every pair but one (Figure 3). Measured on the 69 clinical tasks whose correct action is to write nothing and on all 50 operations tasks, the rate per task falls from 0.165 to 0.003 for GPT-4.1 nano to GPT-5.6 Luna, from 0.081 to 0.003 for GPT-5 nano to GPT-5.6 Luna, from 0.165 to 0.081 between the two older OpenAI models, from 0.151 to 0.022 for Gemma and from 0.106 to 0.028 for Qwen, all significant after Holm correction. Mistral is unchanged (0.077 → 0.088, $p = 0.90$). Because no pair increased significantly, H3, which predicted family-specific directions, is not supported; the exploratory pilot's suggestion that some families became more reckless did not replicate.

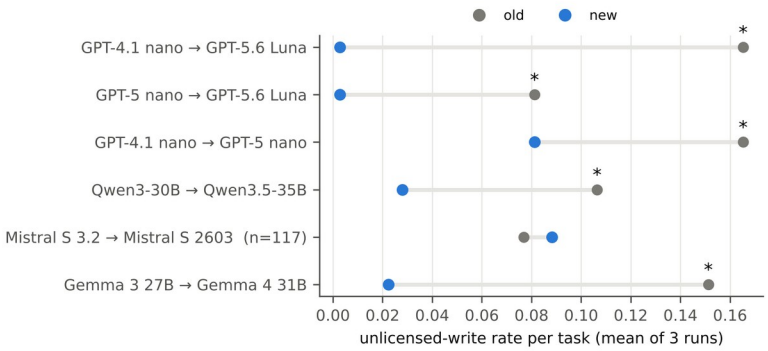


Figure 3. Unlicensed-write rate per task, old and new model; * marks pairs significant after Holm correction.

4.5 Reliability does not erode

H6 predicted that upgrades would turn tasks the old model always passed into flaky ones more often than the reverse. On the two testbeds where it was confirmatory, no cell shows significant erosion; the two smallest one-sided p-values are 0.052 and 0.066, both on MLOps (net losses of 3 and 4 tasks), and neither is close to significant after Holm correction (0.32, 0.33). On the repaired clinical data, where the hypothesis was formed and is exploratory, pass^3 rises in four of six pairs and erosion is never significant. As Section 5 shows, the pattern that motivated H6 was produced by the outage.

5. A silent environment failure and what it would have shown

Repeat-aware evaluation assumes that differences between runs come from the model. This study itself shows how easily that assumption fails. During the confirmatory collection, the clinical record server stopped accepting connections partway through the clinical run and stayed down for roughly ten hours (2026-10-07, about 16:00 UTC, to 2026-10-08, about 01:30 UTC). The environment check at the start of the job had passed, and the harness recorded every tool response verbatim, including `connection refused`. It then graded each episode as usual. Agents that met the error usually reported that the server was unreachable and stopped. The grader scored this as a failure on most tasks but as a success on conditional tasks whose correct action is to order nothing.

I found the problem only when reading the individual episodes behind the one cell with a significant reliable-negative-flip count in my first analysis (Table 3): the provider's official replacement model had "reliably" failed to record a blood-pressure reading on five tasks, and on the first one I opened the server had refused the connection in two of the three runs. A pattern scan of every graded tool response (connection refused, failed new connection, exhausted retries, read time-outs, aborted connections) then found **2,077 of 3,300 clinical episodes (63%)** affected. The share ranged from none for one model, which ran before the outage, through 28% and 42% for the

two newer OpenAI models and 48% and 69% for the two Gemini models, to 84–86% of all episodes for the six open-weight models, which ran almost entirely during it. The operations and MLOps testbeds and the clinical pilot contained no such errors.

The frozen protocol already stated that episodes lost to infrastructure errors are not recorded but re-run. The harness enforced this for errors raised by the model API, but not for errors the environment returned to the agent as ordinary tool output. I quarantined the affected episodes, changed the harness to check the server before every episode and to refuse any episode whose trace contains a connection error, and re-ran them with the same prompts, decoding settings and response cache. All confirmatory results in this paper are computed on the repaired data; the analysis of the contaminated data is kept as an audit record.

What the contaminated data would have shown. Table 3 and Figure 4 compare the clinical results computed on the contaminated data, which is what I would have published had I trusted the job-level check, with the results on the repaired data. Three apparent losses became gains. Gemma 3 → Gemma 4 went from a significant *loss* of 11 points to a significant gain of 13; Qwen3 → Qwen3.5 went from −3 to +11, and GPT-5 nano → GPT-5.6 Luna from −6 to +3. In the other direction, the Mistral step's significant gain of 6 points vanished (−0.3). The only cell with a significant reliable-negative-flip count in the contaminated analysis (five "reliable" breaks for the provider's official replacement model, Holm-corrected $p = 0.006$) vanished; on clean data that cell has none. The drop in pass^3 that motivated hypothesis H6 (Gemma 0.58 → 0.35) became a rise (0.51 → 0.72).

The outage did not simply lower everyone's scores. Most of the affected episodes were still graded, and **43% of them (896 of 2,077) were graded as successes even though the agent could not read the record**. On the two conditional ordering templates (replace magnesium or potassium if the last value is low), 64–65% of blind episodes passed, because doing nothing is the correct action whenever the stored value is normal. Models also differed in how they behaved when blind. One older model submitted orders in 79 of its 252 affected episodes while the record server was refusing its connections; no other model did so in more than 9.2% of its affected episodes. So the outage turned differences in behaviour *under failure* into apparent differences in capability, which is why it reversed upgrade effects instead of merely adding noise.

Table 3. Clinical testbed: upgrade effects on the contaminated and on the repaired data. Holm correction is over all cells of the respective analysis; in the contaminated analysis these include the since-excluded Gemini pair.

Pair	Δ contaminated	Δ clean	RNF contaminated (p Holm)	RNF clean (p Holm)	pass^3 contaminated	pass^3 clean
GPT-4.1 nano → GPT-5.6 Luna	+0.04	+0.18	5 (0.006)	0 (1.0)	0.38 → 0.44	0.38 → 0.48
GPT-5 nano → GPT-5.6 Luna	−0.06	+0.03	0 (1.0)	1 (1.0)	0.38 → 0.44	0.49 → 0.48
GPT-4.1 nano → GPT-5 nano	+0.10	+0.15	2 (0.50)	2 (1.0)	0.38 → 0.38	0.38 → 0.49
Qwen3 → Qwen3.5	−0.03	+0.11	0 (1.0)	4 (0.14)	0.55 → 0.47	0.56 → 0.71
Mistral Small 3.2 → 2603	+0.06	−0.00	0 (1.0)	2 (0.20)	0.41 → 0.44	0.69 → 0.64
Gemma 3 → Gemma 4	−0.11	+0.13	1 (0.78)	4 (0.09)	0.58 → 0.35	0.51 → 0.72

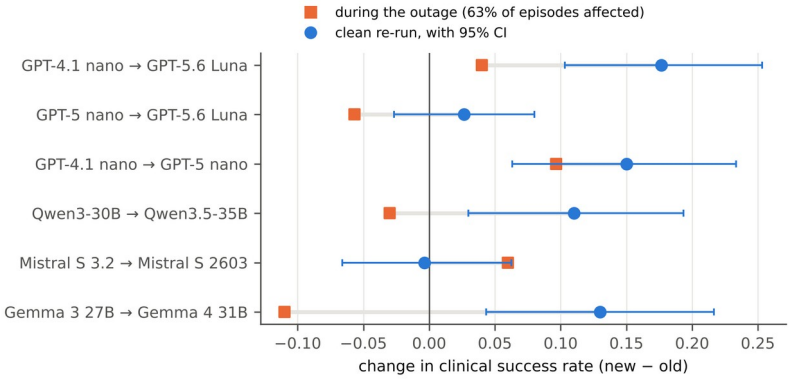


Figure 4. Change in clinical success for each upgrade, computed on data collected during the outage and on the clean re-run.

Lesson. An agent evaluation that grades the final state cannot tell a model failure from an environment failure unless it checks the environment *per episode*, not per job. Run-to-run variation, the very signal a repeat-aware audit uses, is also what a partial outage produces. I release the scan script and recommend that agent benchmarks report, alongside success rates, the number of episodes rejected for environment errors.

6. Limitations

Scale. I study six pairs from four families on 100 clinical tasks, 50 operations tasks and 40 MLOps cases, with three runs per model. This is enough to separate reliable flips from run-to-run churn per cell, but not to estimate small RNF rates precisely; the pooled upper bounds I report are the honest summary. All models are small or “lite” tiers chosen to fit a budget of under ten US dollars of paid inference; larger models may churn less, and my conclusions about noise in one-shot suites apply most directly to the cost-sensitive tiers that teams run at volume.

Three runs. With a task the new model passes with probability one half is classed as an RNF one time in eight. The permutation test accounts for this, but the definition of “reliable” is still coarse. More runs would sharpen it at linear cost; the released episodes allow any reader to recompute every statistic for .

Serving. Each model is pinned to one serving provider with fallbacks disabled, and the provider that answered every call is logged. A different provider, quantisation or decoding stack could change behaviour; for open-weight models my results describe *that model as served by that provider*. Temperature-zero decoding is not deterministic on these services, which is precisely the churn I measure, so exact episodes are not reproducible, but every episode I graded is released.

Testbeds. All three testbeds are code-graded with state-based oracles, which rules out judge error but limits the task types to those whose outcome can be read from a system’s state. The clinical tasks run against a public synthetic record, not real patients; nothing here supports clinical use of any model. The operations and MLOps testbeds are my own constructions on real services and real pipelines, and their task mix shapes absolute success rates.

Free-tier models. A pre-registered seventh pair (Gemini Flash-Lite) ran on a free tier. Its quota of 50 requests per model per day made it impossible to replace the episodes lost to the outage, and it is excluded; my conclusions cover four families.

Rate limits. One provider’s rate limits left 4 of 100 clinical tasks incomplete for one model after three re-run passes; they are excluded from that pair’s clinical cell. Excluding tasks the provider would not serve could bias that cell if the excluded tasks differ systematically, for example by length.

Same-family scope. I study upgrades within a family because that is the migration providers recommend when a model is retired. Cross-family switches are likely to change more, and the pairs studied here do not speak to them.

Hypothesis H6 was added after the clinical data. I state this in the protocol, treat H6 as exploratory on the clinical testbed, and test it as confirmatory only on the operations and MLOps sets, which had not been analysed when it was written.

7. Discussion

What to do at upgrade time. My results suggest three inexpensive habits for a team about to switch an agent to the next model in its family. First, never act on a single failed run: re-run each failure up to twice and flag only tasks that fail every time. In my data this removes all false alarms from one-shot comparisons for about a fifth more episodes, and the cleverer first-action probing I

tried does not do better. Second, check the environment per episode, not per job, and refuse to grade any episode in which a tool returned a connection error; my own per-job check passed while the server was down for ten hours. Third, audit unlicensed writes separately from success. The upgrades I studied mostly reduced them, but this is a property of the pairs I studied rather than a law, and a reduction in overreach is invisible in a success rate that already penalises it.

Why the "broken in places" story did not survive. I began this study expecting, from my pilot and from work on single-turn benchmarks (Cacioli 2026; Sheng and Lu 2026), that same-family upgrades would be better on average but reliably break specific workflows. On clean data they are better on average, often by a wide margin on the operations testbed, and the reliable breaks I see are few and individually indistinguishable from what three runs of the same model produce. Two caveats keep this from being an all-clear. A handful of cells have more reliable negative flips than chance before correction for multiple testing (for example, two open-weight pairs on the clinical testbed), and a team that depends on one of those tasks experiences a real regression whatever the corrected p-value says. And with three runs per model, a reliable flip is a coarse event; a regression that lowers a task's success rate from 90% to 40% will usually not appear as one. Teams with a small set of critical workflows should therefore run those workflows many more times than three, which is cheap when the set is small.

Pre-registration and its limits. Pre-registration did its job twice here. It forced me to report five unmet hypotheses, one of them (H1) missing its criterion only at the boundary, and its exclusion rule, written before any data existed, told me what to do when I found the outage. It did not prevent me from forming a new hypothesis (H6, reliability erosion) on data I did not yet know was corrupted. The confirmatory test of that hypothesis on the two uncorrupted testbeds failed, as did its exploratory version on the repaired clinical data. I report it as a cautionary example: an amendment is only as sound as the data that motivated it.

8. Conclusion

I audited six same-family model upgrades of tool-using agents on three executable testbeds, three runs per model per task, under a pre-registered protocol. At least half of the regressions a one-shot comparison reports are noise, and re-running failures removes them cheaply. Reliable regressions are rare, and newer generations write without licence less often. The largest effect in my study came not from any model but from a silent environment outage, which, before I caught it, had turned three upgrade gains into apparent losses and produced a "reliable" break in a provider's official replacement model. For agent evaluation, checking the environment on every episode matters as much as running each episode more than once.

Declarations

Data and Code Availability: https://github.com/lakshmi-charan/model-upgrade-audit. The repository contains all episodes, serving-provider logs, the per-episode environment check and the analysis code.

Funding: This research was self-funded and received no external funding.

Conflict of Interest: The author declares no competing interests.

Author Contributions: Lakshmi Charan Lingisetty: conceptualization, methodology, software, investigation, analysis, visualization, and writing.

References

Abbassi, Altaf Allah, Leuson Da Silva, Amin Nikanjam, and Foutse Khomh. 2025. “ReCatcher: Towards LLMs Regression Testing for Code Generation.” https://arxiv.org/abs/2507.19390.

Ariño de la Rubia, Eduardo, and Szilard Pafka. 2026. “Identical Runs, Different Results: Benchmarking AI Coding Agents on Open-Weight Models.” https://arxiv.org/abs/2609.33812.

Arviv, Ofir, Kristjan Greenewald, Yotam Perlitz, Hadar Mulian, Michal Shmueli-Scheuer, and Leshem Choshen. 2026. “Stop Guessing When to Stop Testing: Efficient Model Evaluation with Just Enough Data.” https://arxiv.org/abs/2607.08522.

Bansal, Gagan, Besmira Nushi, Ece Kamar, Daniel S. Weld, Walter S. Lasecki, and Eric Horvitz. 2019. “Updates in Human-AI Teams: Understanding and Addressing the Performance/Compatibility Tradeoff.” In *Proceedings of the AAAI Conference on Artificial Intelligence*, 33:2429–37. 01. https://doi.org/10.1609/aaai.v33i01.33012429.

Cacioli, Jon-Paul. 2026. “Beyond the Mean: Within-Model Reliable Change Detection for LLM Evaluation.” https://arxiv.org/abs/2604.27405.

Caldwell, Shane, Max Harley, Ads Dawson, Michael Kouremetis, Vincent Abruzzo, and Will Pearce. 2026. “ScopeBench: Do Agents Preserve Engagement Boundaries Under Goal Pressure?” https://arxiv.org/abs/2609.30325.

Chen, Lingjiao, Matei Zaharia, and James Zou. 2024. “How Is ChatGPT’s Behavior Changing over Time?” *Harvard Data Science Review* 6 (2). https://doi.org/10.1162/99608f92.5317da47.

Chishti, Mohd Sameen, Damilare Peter Oyinloye, and Jingyue Li. 2026. “Test Before You Deploy: Governing Updates in the LLM Supply Chain.” In *Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering (FSE Companion ’26)*, 1584–87. Montreal, QC, Canada: ACM. https://doi.org/10.1145/3803437.3805535.

Dixit, Tanay, Daniel Lee, Sally Fang, Sai Sree Harsha, Anirudh Sureshan, Akash V Maharaj, and Yunyao Li. 2024. “RETAIN: Interactive Tool for Regression Testing Guided LLM Migration.” In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, edited by Delia Irazu Hernandez Farias, Tom Hope, and Manling Li, 301–10. Miami, Florida, USA: Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.emnlp-demo.31.

Echterhoff, Jessica Maria, Fartash Faghri, Raviteja Vemulapalli, Ting-Yao Hu, Chun-Liang Li, Oncel Tuzel, and Hadi Pouransari. 2024. “MUSCLE: A Model Update Strategy for Compatible LLM Evolution.” In *Findings of the Association for Computational Linguistics: EMNLP 2024*, 7320–32. Miami, Florida, USA: Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.findings-emnlp.430.

Good, Phillip. 2005. *Permutation, Parametric, and Bootstrap Tests of Hypotheses*. Third. Springer Series in Statistics. New York: Springer. https://doi.org/10.1007/b138696.

Holm, Sture. 1979. “A Simple Sequentially Rejective Multiple Test Procedure.” *Scandinavian Journal of Statistics* 6 (2): 65–70. https://www.jstor.org/stable/4615733.

Hsu, Chia-Yu, and Shubhanshu Shekhar. 2026. “Efficient Sequential Evaluation of Large Language Models.” https://arxiv.org/abs/2607.17409.

Jiang, Yixing, Kameron C. Black, Gloria Geng, Danny Park, James Zou, Andrew Y. Ng, and Jonathan H. Chen. 2025. “MedAgentBench: A Virtual EHR Environment to Benchmark Medical LLM Agents.” *NEJM AI* 2 (9): AIdbp2500144. https://doi.org/10.1056/AIdbp2500144.

Kim, Hyungjin Lukas. 2026. “When the Model Retires: An Empirical Study of LLM Migration in Open-Source Applications.” https://arxiv.org/abs/2609.31288.

Kossen, Jannik, Sebastian Farquhar, Yarin Gal, and Tom Rainforth. 2021. “Active Testing: Sample-Efficient Model Evaluation.” In *Proceedings of the 38th International Conference on Machine Learning*, edited by Marina Meila and Tong Zhang, 139:5753–63. Proceedings of Machine Learning Research. PMLR. https://proceedings.mlr.press/v139/kossen21a.html.

Kübler, Jonas, Kailash Budhathoki, Matthäus Kleindessner, Xiong Zhou, Junming Yin, Ashish Khetan, and George Karypis. 2026. “When LLMs Get Significantly Worse: A Statistical Approach to Detect Model Degradations.” In *International Conference on Learning Representations (ICLR)*. https://iclr.cc/virtual/2026/poster/10008517.

Lai, Yi-An, Elman Mansimov, Yuqing Xie, and Yi Zhang. 2023. “Improving Prediction Backward-Compatiblility in NLP Model Upgrade with Gated Fusion.” In *Findings of the Association for Computational Linguistics: EACL 2023*, 1010–22. Dubrovnik, Croatia: Association for Computational Linguistics. https://doi.org/10.18653/v1/2023.findings-eacl.74.

Ma, Wanqin, Chenyang Yang, and Christian Kästner. 2024. “(Why) Is My Prompt Getting Worse? Rethinking Regression Testing for Evolving LLM APIs.” In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering – Software Engineering for AI (CAIN 2024)*, 166–71. Association for Computing Machinery. https://doi.org/10.1145/3644815.3644950.

Qu, Yubin, Ying Zhang, Yanjun Zhang, Gelei Deng, Yuekang Li, Leo Yu Zhang, and Yi Liu. 2026. “Overeager Coding Agents: Measuring Out-of-Scope Actions on Benign Tasks.” https://arxiv.org/abs/2605.18583.

Rabanser, Stephan, Sayash Kapoor, Peter Kirgis, Kangheng Liu, Saiteja Utpala, and Arvind Narayanan. 2026. “Towards a Science of AI Agent Reliability.” In *Proceedings of the 43rd International Conference on Machine Learning*, 306:102769–820. Proceedings of Machine Learning Research. PMLR. https://proceedings.mlr.press/v306/rabanser26a.html.

Shawn, Zayx. 2026. “PACE: Anytime-Valid Acceptance Tests for Self-Evolving Agents.” https://arxiv.org/abs/2606.08106.

Sheng, Jia, and Yiwei Lu. 2026. “No Universal Signal Predicts Sample-Level LLM Regression Under Version Updates.” https://arxiv.org/abs/2608.13607.

Shi, Tianneng, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2025. "Progent: Securing AI Agents with Privilege Control." https://arxiv.org/abs/2504.11703.

Vo, Ngoc Phuoc An, Aarya Doshi, and Vadim Sheinin. 2026. "Continuous Process-Level Evaluation for Evolving Enterprise AI Agent Skills." https://arxiv.org/abs/2610.01833.

Wang, Yaxuan, Quan Liu, Zhenting Wang, Zichao Li, Wei Wei, Yang Liu, and Yujia Bao. 2025. "PromptBridge: Cross-Model Prompt Transfer for Large Language Models." https://arxiv.org/abs/2512.01420.

Xie, Yuqing, Yi-An Lai, Yuanjun Xiong, Yi Zhang, and Stefano Soatto. 2021. "Regression Bugs Are in Your Model! Measuring, Reducing and Analyzing Regressions in NLP Model Updates." In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, 6589–6602. Association for Computational Linguistics. https://doi.org/10.18653/v1/2021.acl-long.515.

Yagubyan, Abel. 2026. "How Consistent Are LLM Agents? Measuring Behavioral Reproducibility in Multi-Step Tool-Calling Pipelines." https://arxiv.org/abs/2605.28840.

Yan, Sijie, Yuanjun Xiong, Kaustav Kundu, Shuo Yang, Siqi Deng, Meng Wang, Wei Xia, and Stefano Soatto. 2021. "Positive-Congruent Training: Towards Regression-Free Model Updates." In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 14299–308. https://openaccess.thecvf.com/content/CVPR2021/html/Yan_Positive-Congruent_Training_Towards_Regression-Free_Model_Updates_CVPR_2021_paper.html.

Yao, Shunyu, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2025. "-Bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains." In *International Conference on Learning Representations (ICLR)*, 2025:9965–10017. https://proceedings.iclr.cc/paper_files/paper/2025/hash/1b126cc38b8638e07bef37e7b2bb72bf-Abstract-Conference.html.

Zhang, Taolin, Jiuheng Wan, Hanyu Wang, Tingyuan Hu, and Chengyu Wang. 2026. "OverAct: Measuring and Mitigating Proactive over-Authorization in LLM Tool-Calling Agents." https://arxiv.org/abs/2610.01508.

Zhu, Mingye, Yi Liu, Quan Wang, Junbo Guo, and Zhendong Mao. 2024. "FlipGuard: Defending Preference Alignment Against Update Regression with Constrained Optimization." In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 17333–50. Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.emnlp-main.960.

Appendix A. Pre-registered protocol and amendments

Freeze. Hypotheses H1–H5, the confirmatory task sets, the model list with pinned serving providers, the grading code and the confirmatory analysis script were frozen on 2026-10-07 at 13:40 UTC, before any episode of the confirmatory sets was run. The freeze records a SHA-256 hash of every frozen file. The confirmatory sets are disjoint from the pilot and smoke sets: clinical `main` (100 tasks, 64 of them conditional), operations `main50` (a seeded stratified subsample of 50 tasks, six to seven per workflow template) and MLOps `main40` (a seeded stratified subsample of 40 cases by domain and oracle type; seed 20261007).

Hypotheses as frozen.

- **H1.** Among single-run regressions, the share the new model passes in run 2 or 3 exceeds 50%; pooled over pairs and testbeds, 95% CI from a task-level bootstrap (10,000 resamples); H1 holds if the lower bound exceeds 0.5.
- **H2.** At least one pair × testbed cell has a significant RNF count (within-task permutation test, 10,000 permutations, Holm across cells), and the pooled RNF rate among old-reliable tasks has a 95% upper bound below 10%.
- **H3.** Per pair, a two-sided Wilcoxon signed-rank test of per-task overreach rates (mean over three runs) on clinical tasks with an empty licensed set and on all operations tasks, Holm across pairs. H3 holds if at least two pairs change significantly *and* not all significant changes share a sign.
- **H4.** The procedure that re-runs each failure up to twice before flagging it costs at most 25% more episodes than a single run, on average.
- **H5.** In at least two pairs whose mean success improves (paired bootstrap CI above zero), the RNF count is significant.

Exclusions. Episodes lost to provider or API errors are not recorded and are re-run; a task enters a pair's analysis only when both models have all three runs. Grading uses only code.

Amendment (2026-10-08, 11:12 UTC): H6. After analysing the clinical confirmatory set, and before computing any flip or erosion statistic on the operations or MLOps data, I added H6, *upgrades erode reliability more than they break tasks*. Its statistic per cell is the net loss of always-passing tasks, #(old 3/3, new < 3/3) − #(new 3/3, old < 3/3), with a one-sided within-task permutation test (4,000 permutations, seed 0) and Holm across the cells of a testbed. Because it was formed on clinical data, H6 is exploratory on the clinical testbed and confirmatory on operations and MLOps, where it holds if at least two pairs show significant erosion in at least one of the two testbeds *and*, across all testbeds, erosion-significant cells outnumber RNF-significant cells. The amendment was written into the protocol and frozen together with the new analysis script.

Freeze record. Re-freezing for the amendment overwrote the original freeze file. Comparing file modification times shows that the only frozen files changed since the first freeze are the protocol text (the amendment) and the new transition-analysis script; all code, task sets, configurations and the confirmatory script are unchanged. Later freezes were written to dated files.

Deviations during collection. (i) Two Gemini 2.5 models were removed before the freeze because the provider no longer serves them to new accounts. (ii) Also during the pilot, before the freeze, two open-weight models were re-pinned to a different serving provider (one because its first

provider returned routing errors on every call, one because a free-tier quota interrupted its runs); the affected episodes were quarantined, and none of them enter any analysis. (iii) The Gemini pair ran on a free tier whose quota turned out to be 50 requests per model per day; after the outage below it could not be completed and is excluded from all analyses. (iv) One model's provider rate-limited it during the re-run; after three fill passes 4 of its 100 clinical tasks lacked a complete set of runs and are excluded from that cell, as the frozen rule requires. (v) The protocol text stated that 49 clinical tasks have no licensed write; the frozen analysis code, which defines the set from the live reference solutions, finds 69, and all H3 results use the code's definition.

Outage and enforcement of the exclusion rule (2026-10-08, 17:40 UTC). An audit found that 2,077 of 3,300 clinical episodes had been graded while the record server refused connections (Section 5). These episodes fall under the frozen exclusion rule. They were quarantined, the harness was changed to check the server before each episode and to reject any episode containing a connection error, and the episodes were re-run with unchanged prompts, settings and response cache. The confirmatory analysis was then re-run unchanged. The analysis computed on the contaminated data is superseded and kept as an audit record. H6 had been formed on the contaminated clinical data; its confirmatory test on the operations and MLOps sets, which were unaffected, is unchanged.

Budget. Paid inference cost US$9.87 in total, including the re-run of the contaminated episodes, for which the approved cap was raised from $9.50 to $11.00 before any re-run episode was paid for.

Pilot. A 60-task clinical pilot, run before the freeze and unaffected by the outage, is reported only as exploratory (Appendix B).

Appendix B. Exploratory pilot

Before the freeze I ran 60 clinical tasks (disjoint from the confirmatory set) three times with every model. In the pilot, several open-weight pairs showed significant reliable-negative-flip counts while the OpenAI pairs showed none, and unlicensed writes appeared to fall for some families and rise for others. These observations shaped H2, H3 and H5. None of them replicated on the confirmatory set: no cell is significant after correction, and unlicensed writes fall in every pair that changes significantly. I report the pilot to document how the hypotheses were formed, not as evidence.

Appendix C. Per-cell results

Table 4. Every pair–testbed cell on the repaired data (pairs as in Table 1). Δ is the change in mean success (new − old) with a task-level bootstrap 95% CI; "Old 3/3" counts tasks the old model passed in all runs; single-run regressions are old-3/3 tasks the new model failed in its first run, with the number that passed in run 2 or 3 in parentheses; RNF p-values are from the within-task permutation test, Holm-corrected across the 18 cells; erosion p-values are one-sided, Holm-corrected within each testbed (exploratory on the clinical testbed).

Testbed	Pair	n	Δ [95% CI]	Old 3/3	1-run regr. (vanish)	RNF	p (Holm)	pass^3 old → new	Erosion p (Holm)
Clin.	P1	100	+0.18 [+0.10, +0.25]	38	6 (6)	0	1 (1)	0.38 → 0.48	.994 (1)
Clin.	P2	100	+0.03 [−0.03, +0.08]	49	7 (6)	1	.307 (1)	0.49 → 0.48	.450 (1)
Clin.	P3	100	+0.15 [+0.06, +0.23]	38	5 (3)	2	.100 (1)	0.38 → 0.49	.999 (1)
Clin.	P4	100	+0.11 [+0.03, +0.19]	56	5 (1)	4	.008 (.144)	0.56 → 0.71	1 (1)
Clin.	P5	96	−0.00 [−0.07, +0.06]	66	5 (3)	2	.012 (.197)	0.69 → 0.64	.126 (.757)
Clin.	P6	100	+0.13 [+0.04, +0.22]	51	6 (2)	4	.005 (.088)	0.51 → 0.72	1 (1)
Ops	P1	50	+0.31 [+0.23, +0.41]	18	0 (0)	0	1 (1)	0.36 → 0.92	1 (1)
Ops	P2	50	+0.13 [+0.05, +0.21]	34	2 (2)	0	1 (1)	0.68 → 0.92	.999 (1)
Ops	P3	50	+0.19 [+0.07, +0.31]	18	5 (4)	1	.434 (1)	0.36 → 0.68	1 (1)
Ops	P4	50	+0.67 [+0.57, +0.77]	5	0 (0)	0	1 (1)	0.10 → 0.88	1 (1)
Ops	P5	50	+0.15 [+0.09, +0.22]	25	0 (0)	0	1 (1)	0.50 → 0.84	1 (1)
Ops	P6	50	+0.80 [+0.69, +0.89]	6	0 (0)	0	1 (1)	0.12 → 0.82	1 (1)
MLOps	P1	40	+0.21 [+0.08, +0.35]	6	1 (1)	0	1 (1)	0.15 → 0.20	.933 (1)
MLOps	P2	40	+0.23 [+0.12, +0.33]	3	1 (1)	0	1 (1)	0.07 → 0.20	.989 (1)
MLOps	P3	40	−0.03 [−0.11, +0.06]	6	1 (1)	0	1 (1)	0.15 → 0.07	.052 (.315)
MLOps	P4	40	−0.08 [−0.20, +0.03]	16	5 (4)	1	.091 (1)	0.40 → 0.30	.066 (.328)
MLOps	P5	40	+0.12 [−0.02, +0.28]	3	2 (0)	2	.032 (.454)	0.07 → 0.12	.950 (1)
MLOps	P6	40	+0.08 [−0.07, +0.24]	11	3 (0)	3	.013 (.197)	0.28 → 0.35	.997 (1)